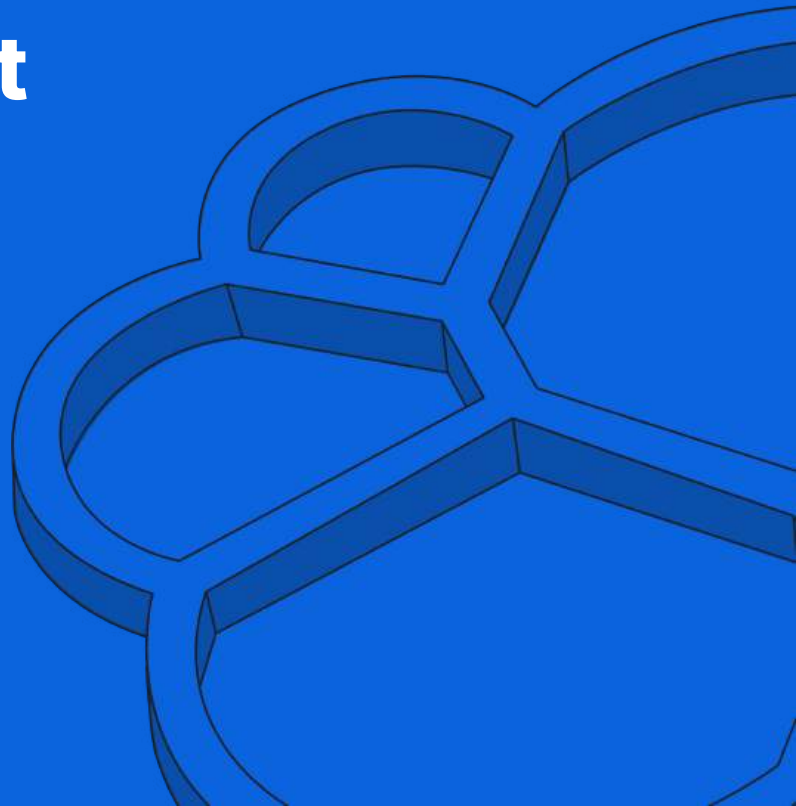


From Diagnosis to Action: Building AI Agents That Fix Problems

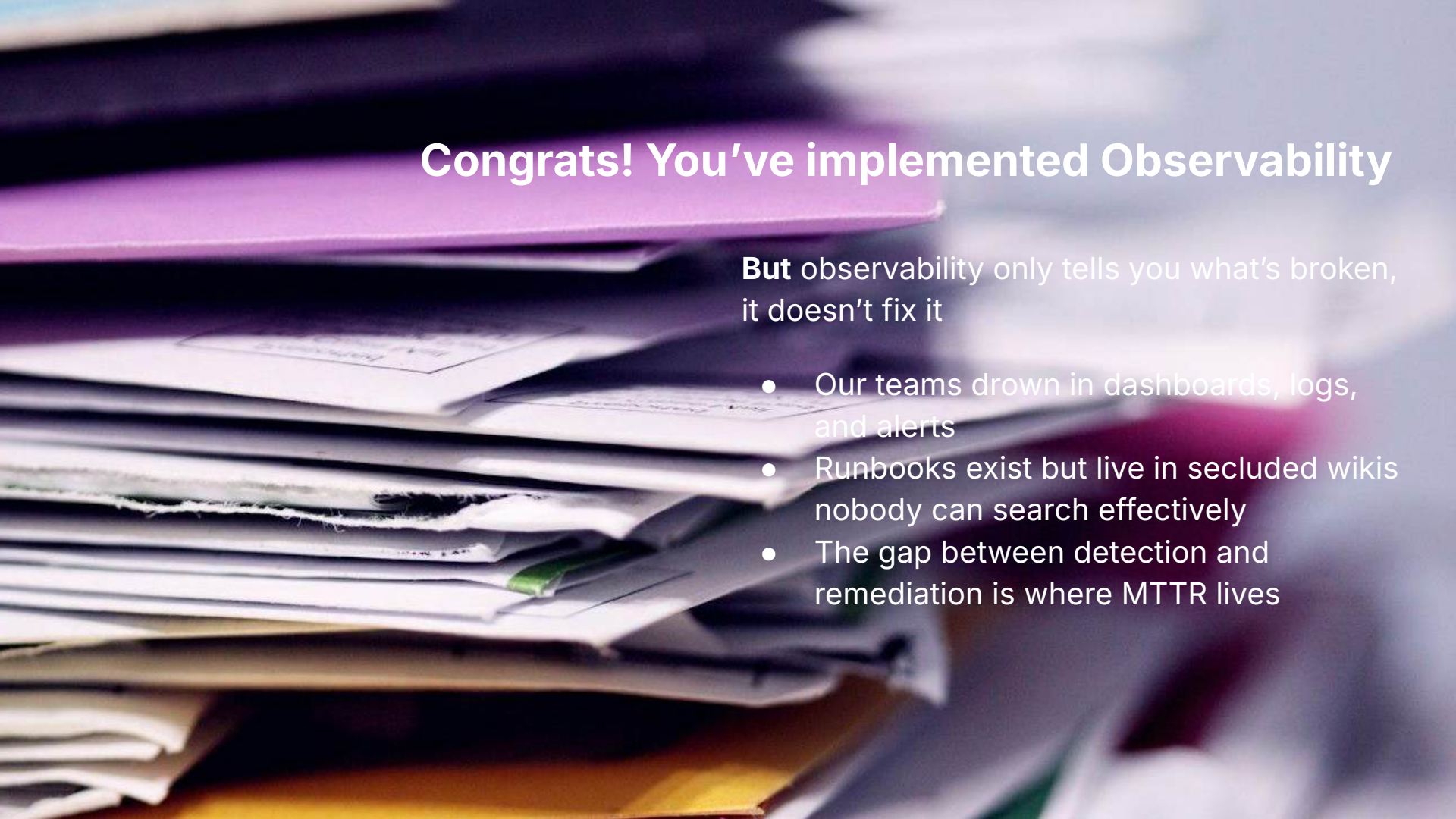
April 16, 2026

Sophia Solomon



Agenda

- | | |
|----|--------------------------------------|
| 01 | Go Mock Services |
| 02 | EDOT Collector Gateway Mode |
| 03 | Agent Builder + Semantic Playbooks |
| 04 | Elastic Workflows: Human-in-the-Loop |



Congrats! You've implemented Observability

But observability only tells you what's broken, it doesn't fix it

- Our teams drown in dashboards, logs, and alerts
- Runbooks exist but live in secluded wikis nobody can search effectively
- The gap between detection and remediation is where MTTR lives

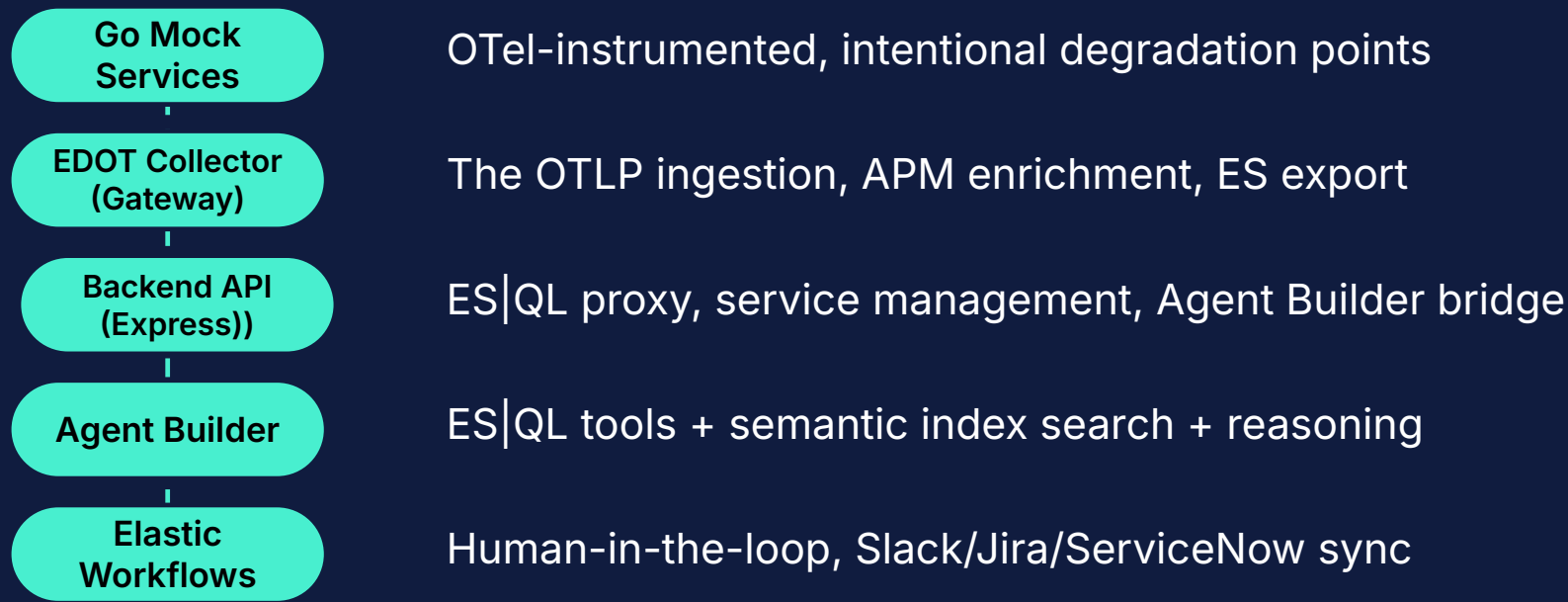
Agents that reason, match and act, but within guardrails

- AI agents that autonomously investigate anomalies using your own telemetry
- Semantic search that matches symptoms to playbooks even with fuzzy descriptions
- Human-in-the-loop workflows that let agents run read-only ops freely, but gate destructive actions



ARCHITECTURE:

Five Layers Along One Pipeline



Go Mock Services

Go Services: Traces, Metrics, and Structured Logs

```
meter :=
otel.Meter(serviceName)
reqCounter, _ :=
meter.Int64Counter("app.requests.total")
errCounter, _ :=
meter.Int64Counter("app.errors.total")
latencyHist, _ :=
meter.Float64Histogram(
    "app.request.duration_ms")
```

```
handler :=
otelhttp.NewHandler(mux,
serviceName,
```

```
otelhttp.WithMessageEvents(
    otelhttp.ReadEvents,
otelhttp.WriteEvents))
```

1. `otelhttp.NewHandler` wraps the mux — automatic spans for every request with method, status, route, duration
2. Custom OTEL metrics:
`app.requests.total`,
`app.errors.total`,
`app.request.duration_ms` histogram
3. Structured JSON logging via slog — `log.level`, `endpoint`, and error context flow into Elasticsearch
4. POST
`/degrade?mode=latency|errors|memory|cpu` toggles failure modes; GET `/health` returns current state + active modes

Four Degradation Modes

Latency Injection

1.5–2.5s randomized, on every request. Simulates upstream dependency timeout. Shows as elevated p99 in APM.

Error Injection

50% of requests return HTTP 500. Simulates database connection exhaustion or downstream failure.

Memory Leak

background goroutine every 2 seconds

CPU Spike

Spins a tight goroutine loop for 100ms per request. Simulates inefficient serialization or GC pressure.

EDOT Collector Gateway Mode

Gateway Pipeline Architecture

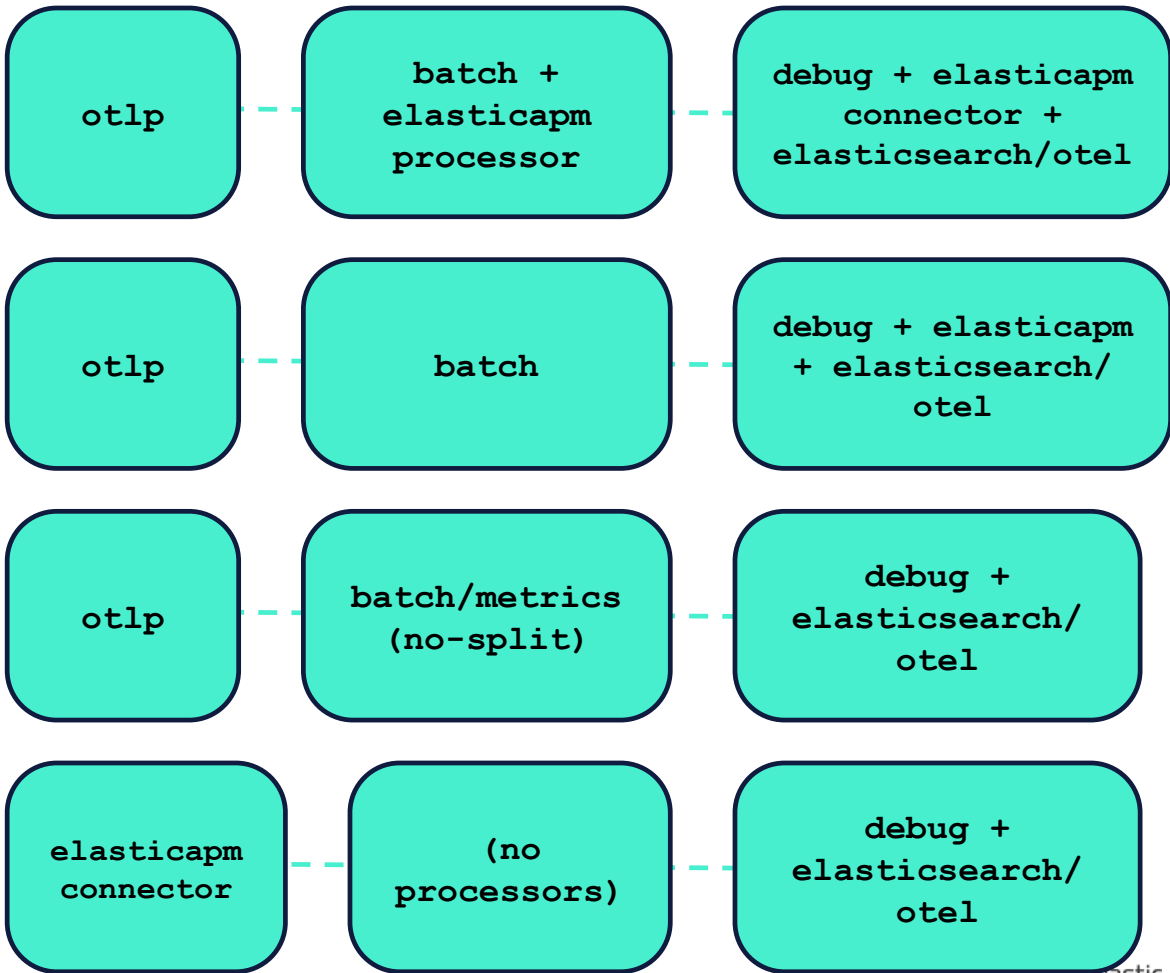
How these four pipelines work together

traces

logs

metrics

metrics/aggregated



Critical Config Decisions

1. batch/metrics sets send_batch_max_size: 0. Keeps metrics batches intact for reliable ingestion into Elasticsearch
2. elasticapm serves 2 roles: it is a processor that enriches spans, and it is a connector that generates aggregated APM metrics from trace data
3. mapping.mode: otel. This preserves OTel-native attributes names, no ECS translation, no data loss
4. The debug exporter on every pipeline is invaluable during development, but needs to be removed for production to reduce verbosity

```
processors:
  batch/metrics:
    send_batch_max_size: 0 #
    Keep batches intact
    timeout: 1s
    elasticapm: {} # Dual:
processor + connector
exporters:
  elasticsearch/otel:
    mapping:
      mode: otel # Preserve
OTel semantics
```

Agent Builder + Semantic Playbooks

Custom ES|QL Tools in Agent Builder

1. Parameterized templates: the agent fills in `?service` and `?window` at runtime
2. Log query: aggregates counts by `log.level` to assess severity distribution
3. Metric Quer: computes AVG and P99 latency from APM data for analysis
4. Error Trend: `DATE_TRUNC` buckets errors into 5-min windows for trend detection

```
FROM logs-*
| WHERE service.name ==
?service
  AND @timestamp > NOW() -
?window
| STATS count = COUNT(*) BY
log.level
| SORT count DESC

FROM metrics-*
| WHERE service.name ==
?service
| STATS avg =
AVG(http.server.duration),
      p99 =
PERCENTILE(http.server.duratio
n, 99)
  BY service.name
```

Semantic Playbook Search

The Index

```
PUT
remediation-playbook
s
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text" },
      "symptoms": {
        "type":
        "semantic_text" },
      "remediation":
      { "type": "text" },
      "severity": {
        "type": "keyword" }
    }
  }
}
```

Why semantic_text?

- Generates embeddings at index time - no separate ML pipeline needed
- Agent describes symptoms in its own words: "service is slow and timing out"
- Your observability backed matches against indexed symptom text even without exact keyword overlap
- "Slow" matches playbooks about "elevated p99 latency"; keyword search alone can't do this
- The index search tool auto-generates queries based on search intent

Agent Reasoning Chain



Agents MUST explain their confidence.

- Elasticsearch's semantic search returns a relevance score: higher means stronger symptom match against the playbook
-
- The demo thresholds this into HIGH vs. MODERATE and adjusts the recommendation: automated remediation or manual review
-
- When using Agent Builder directly, the agent reasons about match quality in its own words, the score may not be numeric, but the explanation matters just as much



Elastic Workflows: Human-in-the-Loop

The Human-in-the-Loop Pattern

✓ Auto-Execute (Read-Only)

- Query error rate via ES|QL
- Check upstream dependency health
- Read service configuration
- Capture heap profile
- Verify connection pool settings

Require Approval (Destructive)

- Killing stale database connections
- Rolling restart of pods
- Scale horizontal replicas
- Reset circuit breakers
- Deploy connection pooler

External System Sync



Slack

Approval requests posted to incident channel. Engineers approve with emoji reactions or thread replies.



Jira

Incident ticket created automatically with playbook steps, confidence score, and affected services.



ServiceNow

Change request opened for destructive actions. Audit trail links back to agent reasoning chain.



Elastic

Full investigation timeline stored in Elasticsearch. Searchable, auditable, feed future training.

Best Practices for AI-Powered Remediation

01

Instrument everything
with OTEL

02

Store playbooks with
semantic_text

03

Parameterize ES|QL
Tools

04

Separate red-only from
destructive

05

Require match quality
explanations
Whether it's a relevance
score or natural language
reasoning, the agent must
show why it chose that
playbook.

06

Keep humans in the
loop, for now



**Thank you so
much for
attending!**